# Funkcje członkowskie typu inline
# Wskaźniki obiektów

Lekcja 14 - Programowanie aplikacji

Klasa 4 technikum - technik programista - kwalifikacja INF.04

```
class Pokoj {
public:
  float dlugosc;
  inline int dlugoscCalc() {
    return round(dlugosc);
  }
};

Pracownik *p = nullptr;
p = &pracownik;
p->wyswietlDane();
```

# 1. Cele lekcji i powiązanie z INF.04

- wyjaśnić, czym jest funkcja członkowska typu inline i kiedy jej użycie ma sens;
- zdefiniować oraz wywołać prostą metodę inline w klasie;
- utworzyć wskaźnik do obiektu, przypisać mu adres obiektu i odwołać się do składowych przez operator ->;
- utworzyć obiekt dynamicznie za pomocą new i poprawnie zwolnić pamięć poleceniem delete;
- porównać obiekt na stosie z obiektem utworzonym dynamicznie na stercie.

**Podstawa programowa**
INF.04.4 Programowanie obiektowe: korzystanie z klas i obiektów, definiowanie pól i metod, tworzenie obiektów oraz posługiwanie się ich składowymi.

**Znaczenie praktyczne**
Lekcja rozwija praktyczne operowanie obiektami w C++ i przygotowuje do dalszych zagadnień: konstruktory, przekazywanie obiektów i zarządzanie pamięcią.

# 2. Funkcja członkowska typu inline - idea

- Funkcja inline to funkcja oznaczona słowem kluczowym inline.
- W podręczniku przedstawiono ją jako funkcję, której kod może zostać wstawiony w miejscu wywołania podczas kompilacji.
- Najlepiej nadają się do tego krótkie, proste metody wykonywane często.
- Metoda inline nadal jest zwykłą funkcją członkowską klasy - ma dostęp do pól obiektu.

**Najprostszy przykład**

```
class Prostokat {
public:
    double a, b;

    inline double pole() {
        return a * b;
    }
};
```

**Uzupełnienie aktualne**

Współczesny C++: słowo inline nie gwarantuje fizycznego „wklejenia” kodu. Decyzję optymalizacyjną podejmuje kompilator.

# 3. Zalecenia dla metod inline

- powinny być krótkie i proste;
- w materiale źródłowym zaleca się unikanie w nich pętli, rozbudowanych instrukcji warunkowych, switch oraz skoków;
- definicja funkcji musi być widoczna przed miejscem jej użycia;
- nie należy stosować inline tylko dlatego, że funkcja jest krótka - czytelność i sens projektu są ważniejsze.

**Ważne doprecyzowanie**

W podręczniku zapisano również, że funkcje inline nie mogą korzystać ze zmiennych statycznych. W nowoczesnym C++ nie jest to zakaz języka - traktujemy tę uwagę jako zalecenie dydaktyczne z materiału źródłowego.

**Praktyka kompilatora**

W praktyce kompilator może sam zdecydować o inliningu nawet bez słowa inline, a może też nie inline’ować funkcji oznaczonej inline.

# 4. Przykład 11.8 - klasa Pokoj

**Definicja klasy i metod inline**

```
#include <iostream>
#include <cmath>
using namespace std;

class Pokoj {
public:
    float dlugosc;
    float szerokosc;

    inline int dlugoscCalc() {
        return round(dlugosc);
    }
    inline int szerokoscCalc() {
        return round(szerokosc);
    }
};
```

**Program główny**

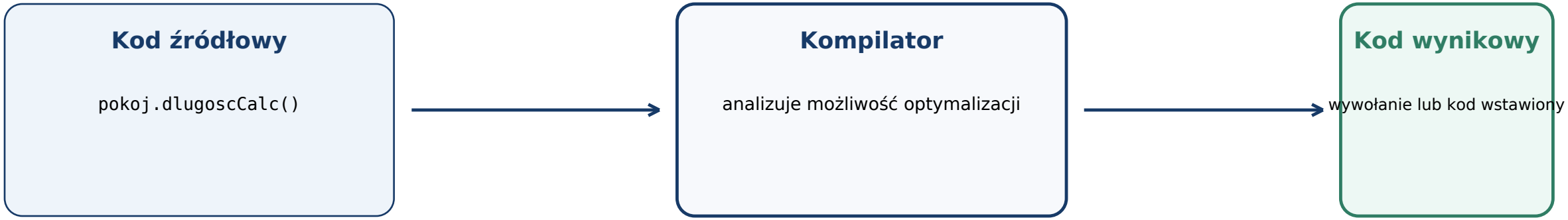
```
int main() {
    Pokoj pokoj;
    pokoj.dlugosc = 1.51;
    pokoj.szerokosc = 2.51;

    cout << pokoj.dlugoscCalc() << endl;
    cout << pokoj.szerokoscCalc() << endl;
    return 0;
}
```

**Co pokazuje przykład?**

Zadanie metod: zaokrąglenie wartości pól dlugosc i szerokosc do najbliższej liczby całkowitej.

# 5. Co dzieje się przy wywołaniu metody inline?




**Wniosek**
Najważniejsza korzyść dydaktyczna: uczeń rozumie, że metoda jest częścią klasy, a jej bardzo krótka definicja może być dostępna bezpośrednio przy deklaracji klasy.

# 6. Ćwiczenie na podstawie 11.8

**Rozbuduj klasę Pokoj o cztery funkcje inline:**

- dlugoscGora() - zaokrąglenie długości w górę;
- szerokoscGora() - zaokrąglenie szerokości w górę;
- dlugoscDol() - zaokrąglenie długości w dół;
- szerokoscDol() - zaokrąglenie szerokości w dół.

**Wzorzec dwóch metod**

```
inline int dlugoscGora() {
    return ceil(dlugosc);
}

inline int dlugoscDol() {
    return floor(dlugosc);
}
```

**Mini-zadanie**

Do wykonania: wywołaj wszystkie cztery metody dla kilku wartości, np. 1.21, 1.50 i 1.89. Zapisz wyniki i porównaj działanie round(), ceil() i floor().

# 7. Wskaźnik do obiektu - podstawowe pojęcia

| Zapis | Znaczenie |
| --- | --- |
| `Pracownik *p;` | deklaracja wskaźnika do obiektu klasy Pracownik |
| `p = nullptr;` | wskaźnik nie wskazuje żadnego obiektu |
| `p = &pracownik;` | przypisanie adresu istniejącego obiektu |
| `p->imie` | dostęp do pola przez wskaźnik |
| `p->wyswietlDane()` | wywołanie metody przez wskaźnik |

# 8. Przykład 11.9 - wskaźnik do obiektu na stosie

**Najważniejszy fragment przykładu**

```
Pracownik pracownik;

Pracownik *w_pracownik = nullptr;
w_pracownik = &pracownik;

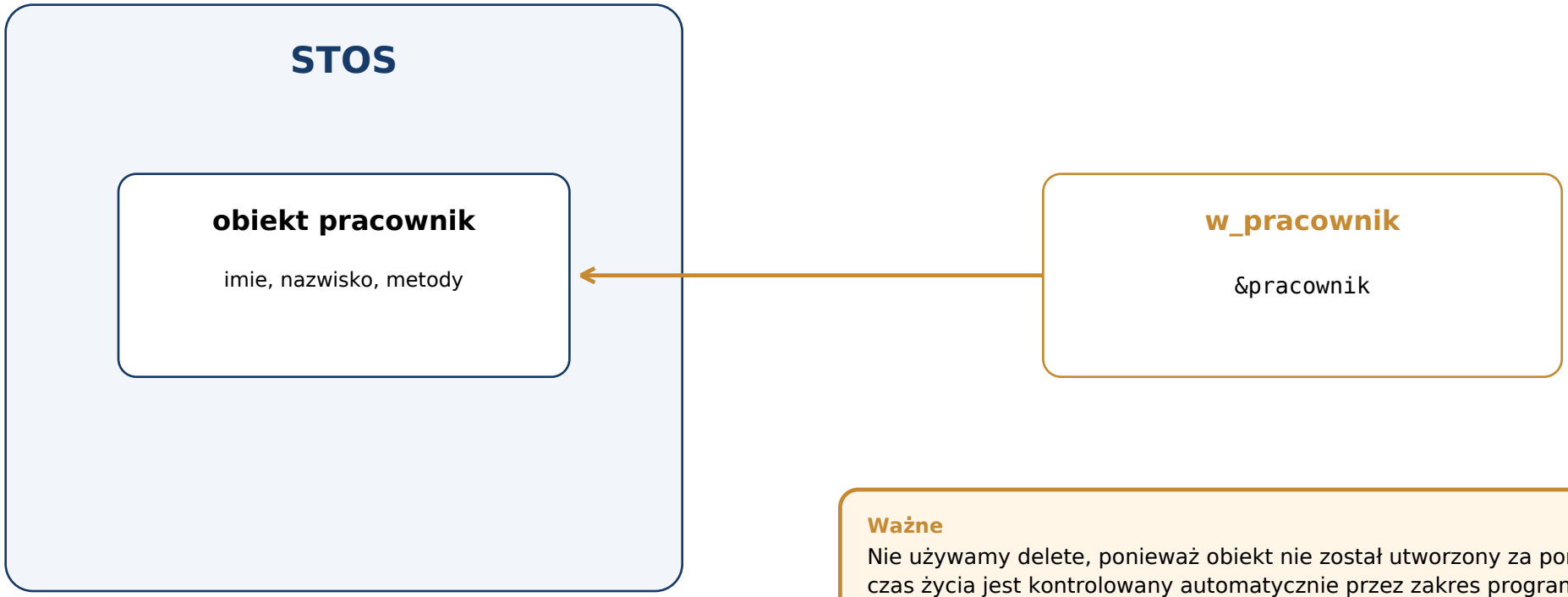
w_pracownik->imie = "Jan";
w_pracownik->nazwisko = "Kowalski";
w_pracownik->wyswietlDane();
```

- obiekt pracownik istnieje na stosie - jego pamięć jest przydzielana automatycznie;
- wskaźnik w_pracownik przechowuje adres obiektu;
- operator & pobiera adres obiektu;
- operator -> wybiera pole lub metodę obiektu wskazywanego przez wskaźnik.

**Operator strzałkowy**

Równoważność: w_pracownik->imie oznacza (*w_pracownik).imie. Zapis z -> jest krótszy i czytelniejszy.

# 9. Stos: obiekt istnieje, wskaźnik tylko go wskazuje




**Ważne**
Nie używamy delete, ponieważ obiekt nie został utworzony za pomocą new. Jego czas życia jest kontrolowany automatycznie przez zakres programu.

# 10. Przykład 11.10 - obiekt dynamiczny na stercie

**Tworzenie i usuwanie obiektu dynamicznego**

```
Pracownik *w_pracownik;

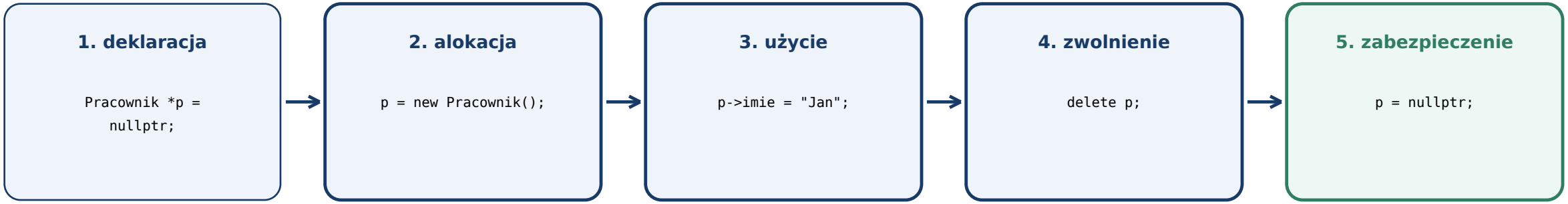
w_pracownik = new Pracownik();

w_pracownik->imie = "Jan";
w_pracownik->nazwisko = "Kowalski";
w_pracownik->wyswietlDane();

delete w_pracownik;
w_pracownik = nullptr;
```

- new tworzy obiekt dynamicznie na stercie i zwraca jego adres;
- wskaźnik przechowuje adres utworzonego obiektu;
- do składowych obiektu nadal odwołujemy się przez ->;
- delete zwalnia pamięć zajętą przez obiekt;
- po delete warto ustawić wskaźnik na nullptr.

# 11. Sterta: cykl życia obiektu dynamicznego




**Dwa typowe zagrożenia**

Jeśli pominiemy delete, powstaje wyciek pamięci. Jeśli po delete spróbujemy użyć starego adresu, możemy otrzymać błąd wynikający z tzw. wiszącego wskaźnika.

# 12. Kropka czy strzałka? Stos czy sterta?

| Sytuacja | Przykład dostępu | Zarządzanie pamięcią |
|---|---|---|
| Zwykły obiekt | pracownik.imie | automatyczne |
| Wskaźnik do istniejącego obiektu | p->imie | obiekt nadal automatyczny |
| Obiekt utworzony przez new | p->imie | wymaga delete |

**Uzupełnienie współczesne**

W nowoczesnym C++ do własności dynamicznych obiektów często preferuje się RAII i inteligentne wskaźniki. Raw new/delete poznajemy tutaj, aby zrozumieć mechanizm pamięci i wskaźników.

# 13. Mini-zadanie na lekcji

## Zaprojektuj klasę Uczen i wykonaj dwa warianty programu:

- pola: imie, nazwisko, klasa, numer_w_dzienniku;
- krótka metoda inline zwracająca numer w dzienniku;
- wariant A: utwórz obiekt na stosie i wskaźnik wskazujący ten obiekt;
- wariant B: utwórz obiekt dynamicznie przez new;
- w obu wariantach odwołaj się do pól przez wskaźnik i wywołaj metodę;
- w wariancie B zwolnij pamięć i ustaw wskaźnik na nullptr.

**Kryterium sukcesu**

Warunek: program ma się kompilować bez błędów i powinien czytelnie pokazywać, kiedy używamy operatora . a kiedy ->.

**Dla chętnych**

Rozszerzenie: dodaj strukturę Data i pole data_urodzenia, analogicznie do ćwiczeń z podręcznika.

# 14. Podsumowanie

- inline oznacza metodę kandydującą do wstawienia w miejscu wywołania, ale decyzję optymalizacyjną podejmuje kompilator;
- krótkie funkcje członkowskie mogą być definiowane bezpośrednio w klasie;
- wskaźnik do obiektu przechowuje adres obiektu;
- operator & pobiera adres, nullptr oznacza brak wskazywanego obiektu, a -> daje dostęp do składowych przez wskaźnik;
- new tworzy obiekt dynamicznie na stercie, delete zwalnia zajętą pamięć;
- po delete warto ustawić wskaźnik na nullptr, aby zmniejszyć ryzyko błędnego użycia starego adresu.

**4 pytania kontrolne**

Sprawdź się: 1) Kiedy użyjesz . zamiast ->? 2) Co robi nullptr? 3) Dlaczego po new potrzebne jest delete? 4) Czy inline gwarantuje wstawienie kodu funkcji?